



Звіт про оригінальність

● Оцінка схожості

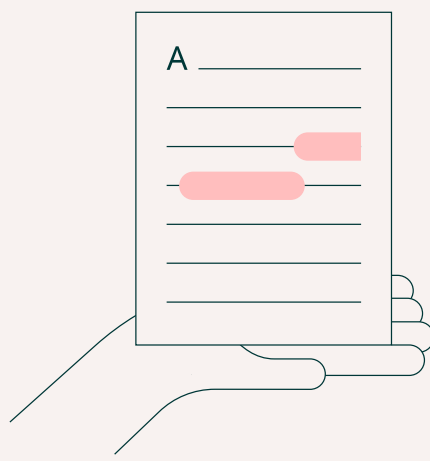
% 8

● Ризик плагіату

СЕРЕДНІЙ

👤 Offiks ⌚ 2026-06-05 22:17

Посилання на звіт: 1aG6h / Посилання користувача: sjWr



Ось вона – Ваша звіт про оригінальність!

Ми раді повідомити, що перевірка вашого документа завершена, і результати вже готові! Наші алгоритми старанно працювали, щоб знайти збіги в наших базах даних.

На наступних сторінках ви знайдете результати перевірки:

Бали

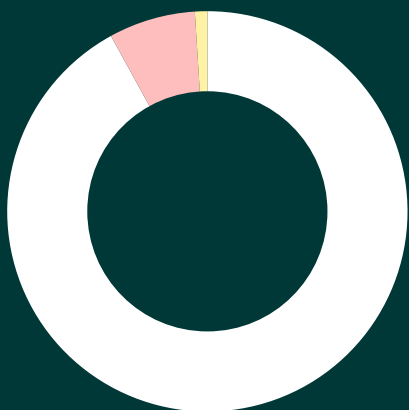
Збіги

Посилання

Ваш документ було перевірено за такими джерелами:

- ☒ База даних інтернет-джерел
- ☐ База даних наукових статей
- ☐ Глибока перевірка (наш вдосконалений алгоритм)

Бали



Збіги тексту	7%
Перефразування	1%
Цитований текст	0%
Неправильне цитування	0%
Збігів не знайдено	92%

Ризик плагіату

СЕРЕДНІЙ

Ризик плагіату вказує, як збіги тексту розподілені по документу. Вищий ризик виникає, коли збіги з'являються близько один до одного, наприклад, у тому самому абзаці або розділі.

Оцінка схожості

Оцінка схожості показує, скільки слів або символів у вашому документі збігаються з текстами інших документів, включаючи перефразовані тексти або неправильні цитати.

% 8

Збіги

6 МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ОДЕСЬКИЙ 11 НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА Факультет математики, фізики та інформаційних
технологій Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА з освітнього компоненту «Програмування» на тему: СТВОРЕННЯ
ІЄРАРХІЇ КЛАСІВ НА ТЕМУ

«Спрощений варіант №20»

Студента 1 курсу, 11 денна форма навчання

11 спеціальність F7 «Комп'ютерна інженерія»

Чебана Вадима Сергійовича

Керівник: к.т.н., доц. Шестопапов С.В.

6 Національна шкала: _____ Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії ____ Антоненко О.С.

3 (підпис) (прізвище та ініціали)

3 ____ 3 Шестопапов С.В.

3 (підпис) (прізвище та ініціали)

3 ____ 3 _____

3 (підпис) (прізвище та ініціали)

3 ОДЕСА 19 - 2026

ЗМІСТ

ВСТУП2

ЗАВДАННЯ4

1. ТЕОРЕТИЧНА ЧАСТИНА5

1.1 Поняття ООП5

1.2Опис об'єктної частини6

2. ПРАКТИЧНА ЧАСТИНА8

2.1 Опис класів8

2.3 Інтерфейс10

ВИСНОВОК14

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ15

3 ВСТУП

3 Мета роботи – створити ієрархію класів на тему «Спрощений варіант № 20» та застосувати її для розв'язання прикладної задачі, визначеної у варіанті завдання. Під час виконання роботи необхідно використовувати основні принципи об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Згідно із завданням потрібно реалізувати систему взаємопов'язаних класів, що описують людину та її характеристики.

Необхідно створити конструктори класів, поля та методи для роботи з об'єктами. Поля класів повинні бути закритими, а доступ до них має здійснюватися через відповідні методи класу. У програмі потрібно реалізувати ієрархію класів на основі відношень успадкування та композиції. Для реалізації динамічного поліморфізму слід використовувати віртуальні методи, абстрактні класи та, за потреби, чисті віртуальні методи.

ЗАВДАННЯ

Базується на четвертій лабораторній ООП.

Взяти клас Person із четвертої лабораторної.

Реалізувати два спадкоємці класу Person. У спадкоємцях перевизначити оператор

вставки в потік або, відповідно, або метод `to_string()`, або метод `print()`.

Створити програмну систему з можливістю додавання, видалення, виведення інформації про екземпляри типу `Person` та його спадкоємців (додатково: пошук). Використовувати вектор вказівників на `Person` (як менш оптимальний варіант припустимо використовувати кілька векторів елементів відповідних класів).

В основній програмі бажано реалізувати гнучкий сценарій роботи програми, наприклад, через текстове меню, щоб користувач міг вибирати свої чергові дії у довільному порядку.

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтоване програмування є методологією розробки, в якій структура програми базується на взаємодії об'єктів. Кожен такий об'єкт є окремим екземпляром конкретного класу, що входять до загальної ієрархії наслідування [1]. Розробник спочатку описує клас (шаблон), а вже **12** під час роботи програми на його основі створюються реальні об'єкти (екземпляри). Існуючі класи можна використовувати як фундамент для нових, які розширюють їхній функціонал, завдяки чому і формується ієрархічна структура.

Будь-яка парадигма програмування спирається на власну теоретичну базу. В основі ОО-стилю лежить об'єктна модель, побудована на трьох ключових концепціях [2,3]:

- а) інкапсуляція;
- б) поліморфізм;
- с) наслідування.

Інкапсуляція – це концепція, яка об'єднує дані та функції (методи) для їхньої обробки в один елемент, захищаючи їх від несанкціонованого зовнішнього доступу або некоректного використання. Таке поєднання компонентів власне і формує об'єкт [1].

Наслідування є процесом, що дозволяє одному класу переймати характеристики та поведінку іншого (базового) класу, доповнюючи їх власними унікальними властивостями, притаманними лише йому [1].

Поліморфізм – принцип, який дає змогу застосовувати однакове ім'я для виконання схожих за змістом, але різних за технічною реалізацією завдань (тобто використання одного інтерфейсу для загального класу дій). На практиці це виражається у здатності об'єкта автоматично обирати потрібний внутрішній метод залежно від типу вхідних

даних [1].

1.2 Опис об'єктної частини

Програма пропонує інформаційну систему обліку та керування даними, яка побудована на базі об'єктно-орієнтованої ієрархії із застосуванням принципів успадкування, інкапсуляції та поліморфізму. Основою архітектури є базовий клас `Person`, який описує людину та містить її загальні анкетні характеристики: прізвище, ім'я, країну проживання, а також контактні й фізичні параметри. Можливість виконання тих чи інших дій у програмі або коректність збереження сутностей безпосередньо залежить від вбудованої валідації даних: властивості класу контролюють, щоб зріст та вага не набували від'ємних значень, а імена не залишалися порожніми. Для забезпечення динамічного керування та підтримки поліморфної поведінки всі екземпляри базового та похідних класів зберігаються в єдиній колекції – динамічному списку об'єктів базового типу (`List<Person>`).

Звичайний користувач системи взаємодіє з базовим класом `Person` та двома його прямими спадкоємцями – `Student` та `Employee`. Спадкоємці повністю перебирають на себе базовий функціонал, але адаптують його під конкретні категорії осіб:

Клас `Student` (Студент) додатково оперує логікою навчального процесу, авторизується за назвою університету та поточним курсом навчання.

Клас `Employee` (Працівник) отримує доступ до розширених полів професійної діяльності, де зберігаються дані про компанію (місце роботи) та займану посаду.

Усі три класи перевизначають базовий метод `ToString()`, що дозволяє автоматично виводити повну інформацію про об'єкт у потік відповідно до його фактичного типу.

Клас `Menu` та головна програма реалізують гнучкий сценарій роботи через текстове меню, щоб користувач міг вибирати свої чергові дії у довільному порядку. Меню пропонує користувачеві чітко розподілений функціонал: перегляд повної бази, додавання нових людей (відповідно до трьох категорій) із заповненням усіх властивостей, а також видалення об'єктів. Окремим важливим елементом є режим пошуку за прізвищем, який не лише знаходить збіги у списку `database`, а й проводить інтегрований аналіз характеристик людини — автоматично перевіряє за допомогою методів `IsHeightOver200()` та `LivesInPoland()`, чи перевищує зріст особи 200 см і чи проживає вона на території Польщі.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

1) Клас «Person»:

a) Приватні поля: firstName, lastName, height, weight, country, phoneNumber.

b) Конструктори:

– Конструктор без параметрів.

– Конструктор з параметрами.

c) Методи:

– IsHeightOver200 – аналіз зросту, повертає "Так", якщо зріст особи перевищує 200 см.

– LivesInPoland – аналіз географічного положення, повертає "Так", якщо країна проживання особи дорівнює "Польща" або "Poland".

– CheckAndFixPhoneNumber – внутрішній метод автоматичного виправлення формату номера: якщо телефон починається з "0", додає "+38".

– Перевизначення ToString – повертає відформатований інформаційний рядок про базові дані людини.

2) Клас «Student», нащадок класу «Person»:

a) Автоматичні властивості (поля): University, CourseYear.

b) Конструктори:

– Конструктор без параметрів.

– Конструктор з параметрами.

c) Методи:

– Перевизначення ToString – повертає повну інформацію про об'єкт, доповнюючи базові дані людини академічними атрибутами студента.

3) Клас «Employee», нащадок класу «Person»:

a) Автоматичні властивості (поля): Company, Position.

b) Конструктори:

– Конструктор без параметрів.

– Конструктор з параметрами

c) Методи:

– Перевизначення ToString – повертає сформований рядок з інформацією про об'єкт, поєднуючи особисті дані з назвою компанії та займаною посадою.

4) Статичний клас «Menu»:

a) Приватні поля: database

b) Методи:

– InitDatabase – заповнює колекцію database початковими тестовими записами для кожного типу об'єктів (Person, Student, Employee).

– PrintAll – перевіряє базу на наявність елементів та виводить у консоль список усіх зареєстрованих людей, використовуючи поліморфний виклик методу ToString().

– AddPerson – універсальний інтерактивний метод, який зчитує дані з консолі та додає до списку новий об'єкт (Person або одного з нащадків) залежно від переданого типу (type).

– RemovePerson – виконує пошук за прізвищем користувача серед елементів колекції та, у разі успішного збігу, повністю видаляє знайдений запис із бази.

– SearchPerson – здійснює пошук усіх людей із заданим прізвищем, виводить їхні повні картки та запускає інтегровані методи аналізу щодо зросту (IsHeightOver200) і місця проживання (LivesInPoland).

5) Клас «Program»:

a) Методи:

– Main – точка входу в програму. Встановлює кодування UTF-8 для коректного відображення символів у консолі, ініціалізує початкову базу даних та запускає нескінченний цикл while(isRunning). Цикл відображає інтерактивне текстове головне меню, обробляє вибір користувача через конструкцію switch(choice) із викликом відповідних методів класу Menu, а також здійснює глобальний перехоплення винятків try-catch для захисту від критичних помилок введення.

2.2 Інтерфейс

Готовий проєкт розміщено в [4]. Під час запуску програми користувач бачить головне вікно з інтерактивним консольним меню (Рис. 2.1).

Рисунок 2.1 – Головне меню програми

Цей екран є центральним вузлом управління системою. Користувачеві пропонується ввести цифру від 0 до 6 для вибору необхідної операції. Якщо обрати пункт «1», програма звертається до методу `Menu.PrintAll()` та виводить на екран повний список усіх людей, що наразі зареєстровані в базі даних (Рис. 2.2).

Рисунок 2.2 – Виведення списку всіх людей

Для розширення бази даних користувач може скористатися пунктами меню 2, 3 або 4. Залежно від вибору, система очікує послідовне введення загальних анкетних характеристик особи, а також специфічних полів для профільних класів (Рис. 2.3).

Рисунок 2.3 – Вікно додавання нового Студента

У разі вибору пункту «5» користувач може видалити будь-яку особу із загальної колекції. Для цього система просить ввести прізвище. Пошуковий алгоритм знаходить перший збіг у базі даних (без урахування регістру літер) та безпосередньо видаляє об'єкт (Рис. 2.4).

Рисунок 2.4 – Процес видалення запису за прізвищем

Пункт головного меню «6» активує режим розширеного пошуку та інтегрованого аналізу даних. Користувач вводить прізвище особи, після чого програма не лише виводить повну картку знайденого об'єкта завдяки поліморфному методу `ToString()`, а й автоматично проводить аналітичну перевірку фізичних та географічних параметрів за допомогою методів `IsHeightOver200()` та `LivesInPoland()` (Рис. 2.5).

Рисунок 2.5 – Вікно пошуку та аналізу особи за прізвищем

Вся система захищена від некоректного введення даних (наприклад, від'ємного зросту чи порожнього імені) за допомогою блоку `try-catch` у файлі `Program.cs` та механізму генерації винятків `ArgumentException` всередині властивостей класу `Person`. У разі помилки програма виведе повідомлення про причину і повернеться до головного меню. Для безпечного завершення роботи та закриття консольного додатка користувачеві достатньо ввести символ «0».

ВИСНОВОК

Під час виконання курсової роботи було розроблено консольну програму на мові програмування C# для автоматизації обліку даних про різні категорії людей. Програма побудована з використанням об'єктно-орієнтованого підходу, 10 що дозволило створити гнучку та масштабовану архітектуру.

10 У ході роботи було реалізовано ієрархію класів, де базовий клас Person містить загальні антропометричні та контактні дані, а похідні класи Student та Employee розширюють цей функціонал специфічними властивостями (місце навчання, посада тощо). Особливу увагу приділено принципу інкапсуляції: доступ до полів класу здійснюється через властивості з інтегрованою валідацією даних, що запобігає появі логічних помилок (наприклад, введення від'ємного зросту чи порожнього імені).

Для управління базою даних використано динамічну колекцію List<Person>, яка завдяки механізму поліморфізму **17** дозволяє зберігати об'єкти різних типів в одному списку та коректно опрацьовувати їх за допомогою віртуальних методів. Розроблений інтерфейс у вигляді текстового меню забезпечує зручну взаємодію з користувачем, дозволяючи виконувати такі операції:

- а) додавання нових записів різних типів;
- б) перегляд повної бази даних;
- в) видалення та пошук інформації за прізвищем;
- г) проведення автоматизованого аналізу параметрів (перевірка країни проживання та фізичних показників).

Програма пройшла тестування на різних наборах даних, продемонструвавши стабільну роботу та коректну обробку виняткових ситуацій за допомогою блоків try-catch. Результати роботи підтвердили

ефективність використання об'єктно-орієнтованих технологій для створення прикладного програмного забезпечення.

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

Грицюк Ю. І., Рак Т. Є. **7** Об'єктно-орієнтоване програмування мовою C#. Навчальний посібник. – Львів: Вид-во ЛДУ БЖД, 2024. – 360 с.

Фрімен А. C# для професіоналів. Тонкощі програмування / А. Фрімен. – Київ: Діалектика, 2025. – 850 с.

Специфікація мови C#: класи, структури та записи [Електронний ресурс] // Microsoft Learn. – Режим доступу: <https://learn.microsoft.com/uk-ua/dotnet/csharp/fundamentals/types/classes>

Репозиторій GitHub на курсовий – Режим доступу: <https://github.com/Nineteeeeeen/Kursach>

Додаток А. 8 Діаграма класів

8 Рисунок А1. – Діаграма класів

8 Додаток Б. Код класу

```
using System;

namespace ConsoleApp1
{
    4 public class Person
    4 {

        private string firstName;

        private string lastName;

        4 private int height;

        4 private int weight;

        4 private string country;

        private string phoneNumber;

        public Person() : this("не вказано", "не вказано", 0, 0, "не вказано", "не вказано") { }

        public Person(string 16 firstName, string lastName, int height, int weight, string country,
            string phoneNumber)

        {

            FirstName = firstName;

            LastName = lastName;

            Height = height;

            Weight = weight;

            Country = country;

            PhoneNumber = phoneNumber;
```

```
CheckAndFixPhoneNumber();

}

public string FirstName
{
    get { return firstName; }

    set
    {
        if (string.IsNullOrEmpty(value))
        {
            throw new ArgumentException("Прізвище не може бути порожнім");
        }

        firstName = value;
    }
}

public string LastName
{
    get { return lastName; }

    set
    {
        if (string.IsNullOrEmpty(value))
        {
            throw new ArgumentException("Ім'я не може бути порожнім!");
        }

        lastName = value;
    }
}
```

```
}  
  
}  
  
public int Height  
  
{  
  
get { return height; }  
  
set
```

Лістинг Б.1 – клас Person, аркуш 1

```
if (value < 0 || value > 300)  
  
{  
  
throw new ArgumentException("Зріст повинен бути в діапазоні від 0 до 300 см");  
  
}  
  
height = value;  
  
}  
  
}  
  
public int Weight  
  
{  
  
get { return weight; }  
  
set  
  
{  
  
if (value < 0)  
  
{  
  
throw new ArgumentException("Вага 15 повинна бути більшою за 0");  
  
}  
  
}
```

```
weight = value;

}

}

public string Country
{
    get { return country; }

    set
    {
        if (string.IsNullOrEmpty(value))
        {
            throw new ArgumentException("Країна проживання не може бути порожньою");
        }

        country = value;
    }
}

public string PhoneNumber
{
    get { return phoneNumber; }

    set
    {
        if (string.IsNullOrEmpty(value))
        {
            throw new ArgumentException("Номер телефону не може бути порожнім");
        }
    }
}
```

```

phoneNumber = value;

}

}

public string IsHeightOver200()
{
    if (height > 200)
    {
        return "Так";
    }
    else
    {
        return "Hi";
    }
}

public string LivesInPoland()
{
    string lowerCountry = country.ToLower();

    if (lowerCountry == "польща" || lowerCountry == "poland")
    {
        return "Так";
    }
    else
    {

```

Лістинг Б.1 – клас Person, аркуш 2


```

return "Hi";

}

}

public void CheckAndFixPhoneNumber()

{

if (!string.IsNullOrEmpty(phoneNumber) && phoneNumber.StartsWith("0"))

{

phoneNumber = "+38" + phoneNumber;

}

}

1 public override string ToString()

1 {

return $"[Людина] Прізвище: {FirstName}, Ім'я: {LastName}, Зріст: {Height}см, Вага: {Weight}кг, Країна: {Country}, Тел: {PhoneNumber}";

}

}

}

```

Лістинг Б.1 – клас Person, аркуш 3

```

1 using System;

1 using System.Collections.Generic;

2 using 2 System.Text;

namespace ConsoleApp1

{

5 public class Student : Person

```

```

{

public string University { get; set; }

public int CourseYear { get; set; }

public Student() : base()

{

University = "не вказано";

CourseYear = 1;

}

5 public Student(string firstName, string lastName, int height, int weight, string country,
string phoneNumber, string university, int courseYear) : base(firstName, lastName, height,
weight, country, phoneNumber)

{

University = university;

CourseYear = courseYear;

}

1 public override string ToString()

1 {

return $"[Студент] Прізвище: {FirstName}, Ім'я: {LastName}, Зріст: {Height}см, Вага:
{Weight}кг, Країна: {Country}, Тел: {PhoneNumber}, Університет: {University}, Курс:
{CourseYear}";

}

}

}

```

Лістинг Б.2 – клас 14 Student.

```
14 using System;
```

```
14 using System.Collections.Generic;
```

```
2 using 2 System.Text;
```

```
namespace ConsoleApp1
```

```
{
```

```
1 public class Employee : 1 Person
```

```
{
```

```
18 public string Company { 18 get; set; }
```

```
2 public string Position { get; set; }
```

```
public Employee() : base()
```

```
{
```

```
Company = "не вказано";
```

```
Position = "не вказано";
```

```
}
```

```
public Employee(string 2 firstName, string lastName, int height, int weight, string country,  
string phoneNumber, string 2 company, string position) : base(firstName, lastName, height,  
weight, country, phoneNumber)
```

```
{
```

```
Company = company;
```

```
Position = position;
```

```
}
```

```
1 public override string ToString()
```

```
1 {
```

```
return $"[Працівник] Прізвище: {FirstName}, Ім'я: {LastName}, Зріст: {Height}см, Вага:  
{Weight}кг, Країна: {Country}, Тел: {PhoneNumber}, Компанія: {Company}, Посада:  
{Position}";
```

```
}  
  
}  
  
}
```

Лістинг Б.3 – клас Employee.

```
1 using System;  
  
1 using System.Collections.Generic;  
  
13 using System.Linq;  
  
13 namespace ConsoleApp1  
  
13 {  
  
1 public static class Menu  
  
{  
  
private static List<Person> database = new List<Person>();  
  
2 public static void InitDatabase()  
  
{  
  
database.Add(new Person("Шевченко", "Тарас", 185, 80, "Україна", "+380991234567"));  
  
database.Add(new Student("Франко", "Іван", 175, 70, "Україна", "+380501112233",  
"Мечникова", 3));  
  
database.Add(new Employee("Міцкевич", "Ілля", 205, 90, "Польща", "+380679998877", "IT-  
Corp", "Розробник"));  
  
}  
  
2 public static void PrintAll()  
  
{  
  
if (database.Count == 0)  
  
{  
  
Console.WriteLine("База даних порожня.");
```

```
return;
```

```
}
```

```
Console.WriteLine("Список записів:");
```

Лістинг Б.4 – клас Menu, аркуш 1

```
foreach (Person person in database)
```

```
{
```

```
    Console.WriteLine(person.ToString());
```

```
}
```

```
}
```

```
public static void AddPerson(int type)
```

```
{
```

```
    Console.Write("Введіть прізвище: ");
```

```
    string fName = Console.ReadLine() ?? "";
```

```
    Console.Write("Введіть ім'я: ");
```

```
    string lName = Console.ReadLine() ?? "";
```

```
    Console.Write("Введіть зріст (см): ");
```

```
    int height = int.Parse(Console.ReadLine() ?? "0");
```

```
    Console.Write("Введіть вагу (кг): ");
```

```
    int weight = int.Parse(Console.ReadLine() ?? "0");
```

```
    Console.Write("Введіть країну проживання: ");
```

```
    string country = Console.ReadLine() ?? "";
```

```
    Console.Write("Введіть номер телефону: ");
```

```
    string phone = Console.ReadLine() ?? "";
```

```
    if (type == 1)
```

```

{
    database.Add(new Person(fName, lName, height, weight, country, phone));
}

9 else if (type == 2)
{
    Console.Write("Введіть Університет: ");
    string uni = Console.ReadLine() ?? "";
    Console.Write("Введіть курс: ");
    int course = int.Parse(Console.ReadLine() ?? "1");
    database.Add(new Student(fName, lName, height, weight, country, phone, uni, course));
}

9 else if (type == 3)
{
    Console.Write("Введіть Компанію: ");
    string comp = Console.ReadLine() ?? "";
    Console.Write("Введіть Посаду: ");

    string pos = Console.ReadLine() ?? "";
    database.Add(new Employee(fName, lName, height, weight, country, phone, comp, pos));
}

Console.WriteLine("Запис успішно додано!");
}

public static void RemovePerson()
{

```

```

Console.Write("Введіть прізвище для видалення: ");

string targetName = Console.ReadLine() ?? "";

Person personToRemove = null;

foreach (Person p in database)

{

if (p.FirstName.ToLower() == targetName.ToLower())

{

personToRemove = p;

break;

}

}

if (personToRemove != null)

{

Лістинг Б.4 – клас Menu, аркуш 2

database.Remove(personToRemove);

Console.WriteLine("Запис видалено.");

}

else

{

Console.WriteLine("Нікого не знайдено.");

}

}

public static void SearchPerson()

{

```

```

Console.Write("Введіть прізвище для пошуку: ");

string targetName = Console.ReadLine() ?? "";

List<Person> results = new List<Person>();

foreach (Person p in database)
{
    if (p.FirstName.ToLower() == targetName.ToLower())
    {
        results.Add(p);
    }
}

if (results.Count > 0)
{
    Console.WriteLine($"Знайдено {results.Count} запис(ів):");

    foreach (Person p in results)
    {
        Console.WriteLine(p.ToString());

        Console.WriteLine($" Зріст > 200 см? {p.IsHeightOver200()}");

        Console.WriteLine($" Живе у Польщі? {p.LivesInPoland()}");
    }
}

else
{
    Console.WriteLine("Нікого не знайдено.");
}

```


}

}

}

Лістинг Б.4 – клас Menu, аркуш 3

Посилання

Це джерела виділених збігів у вашому документі. Кожен збіг позначено темно-зеленим числом, яке відповідає вказаному тут джерелу. Джерела впорядковані за схожістю — чим вищий бал, тим сильніше збіг.

#	Джерело	%
1	cad.kpi.ua	1.2%
2	medium.com	0.8%
3	ua-referat.com	0.7%
4	java-programming.mooc.fi	0.4%
5	stackoverflow.com	0.4%
6	essuir.sumdu.edu.ua	0.4%
7	fpm.dnu.dp.ua	0.4%
8	iq.vntu.edu.ua	0.3%
9	atep.kpi.ua	0.3%
10	org2.knuba.edu.ua	0.3%
11	BiblioFond.ru	0.3%
12	wayscience.com	0.2%
13	openarchive.nure.ua	0.2%
14	dstu.dp.ua	0.2%
15	detector.media	0.2%
16	geeksforgeeks.org	0.2%
17	essuir.sumdu.edu.ua	0.2%
18	sfdc99.com	0.2%
19	dspace.onu.edu.ua	0.1%



Дякуємо, що перевірили
свій документ за допомогою
Plag!